

Order Complexity Analysis

Amount of space or Time taken up by an algorithm/code as function of input size.

NOT the actual time taken.

Ex- Case-I (worst case)
given array unsorted

2	5	4	1	3	8
---	---	---	---	---	---

Apply Linear Search
to find max element

- Compare each element with max element, perform multiple operation.

{ than mean
when we increase
the size of array
automaticall
increase the
time

$T \propto n$

Case-II

given Sorted Array

1	2	3	4	5	6
---	---	---	---	---	---

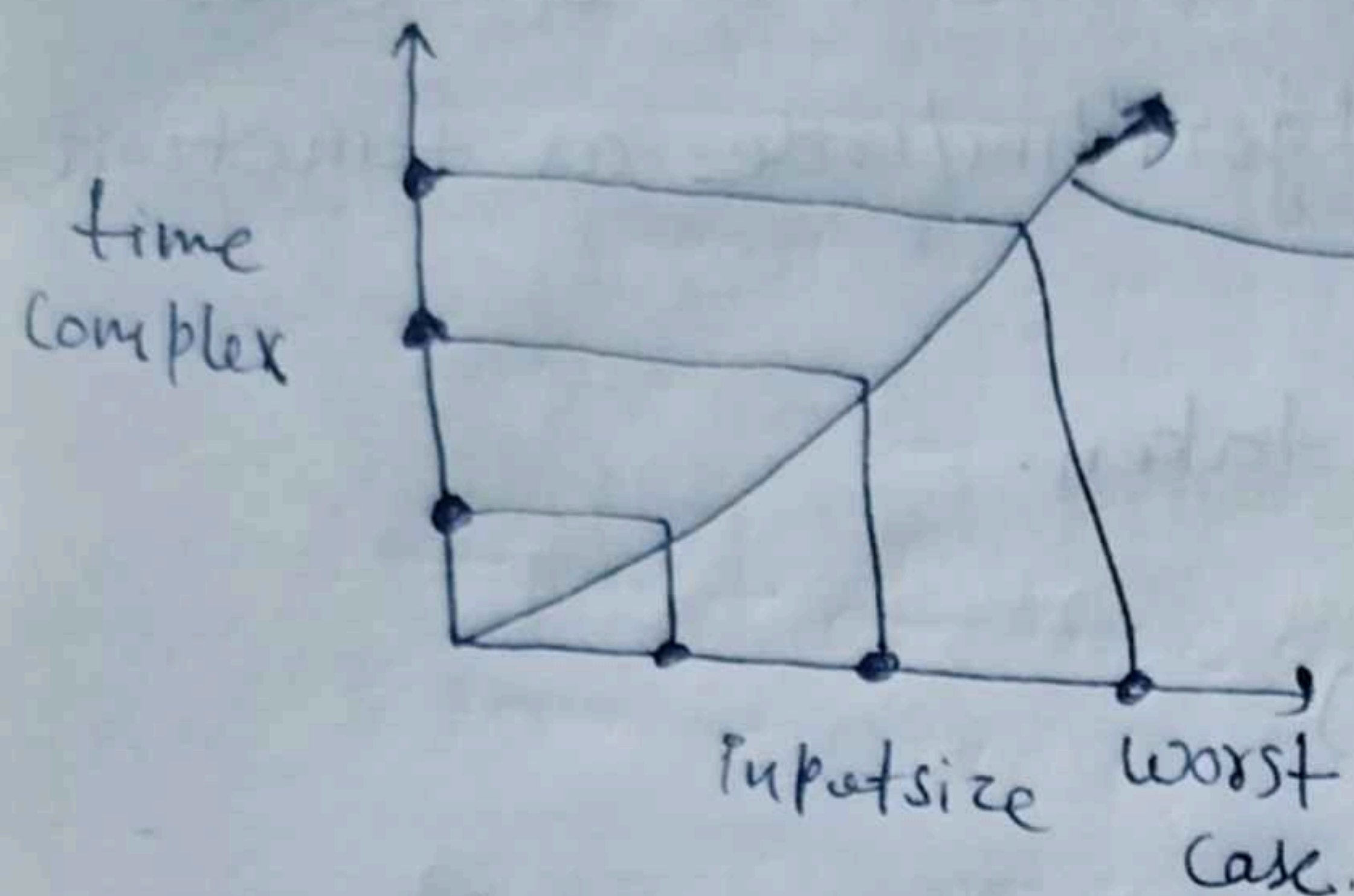
found max element, largest = $arr[n-1]$

In this case time is constant because when we increase the size of array, don't affect the time because we have only one operation that is perform.

$n \uparrow$, T is constant.

Linear Search

Worst Case

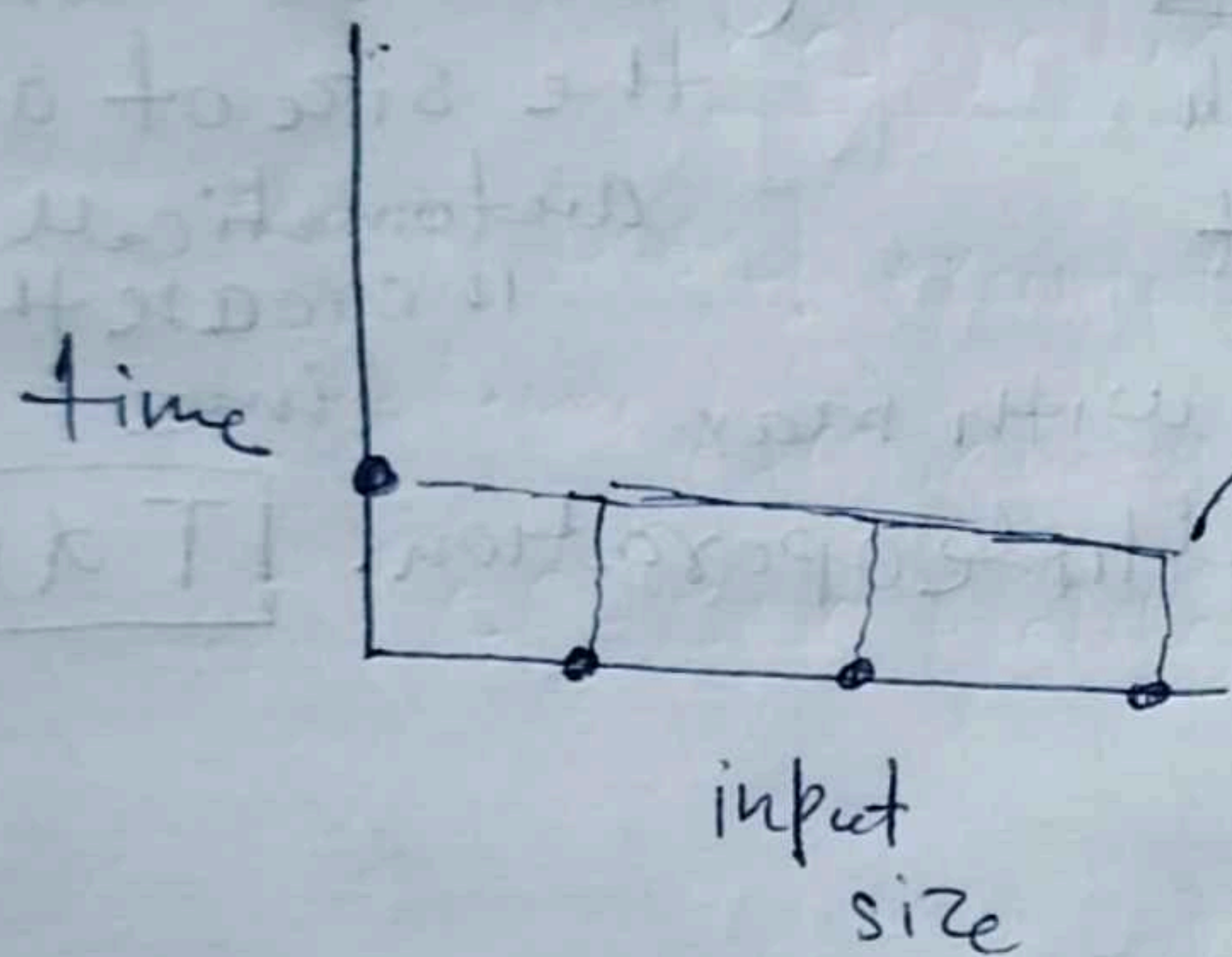


$$y = ax + b$$

$$\text{time} = an + b \text{ (ignore constant)}$$

$$\text{time} = O(n)$$

when time complexity is constant



$$T_c = O(1)$$

constant time complexity

Big O Notation (Upper bound)

We always try to find worst case complexity

• Using Graph

Steps

- 1 ignore constants
- 2 largest term

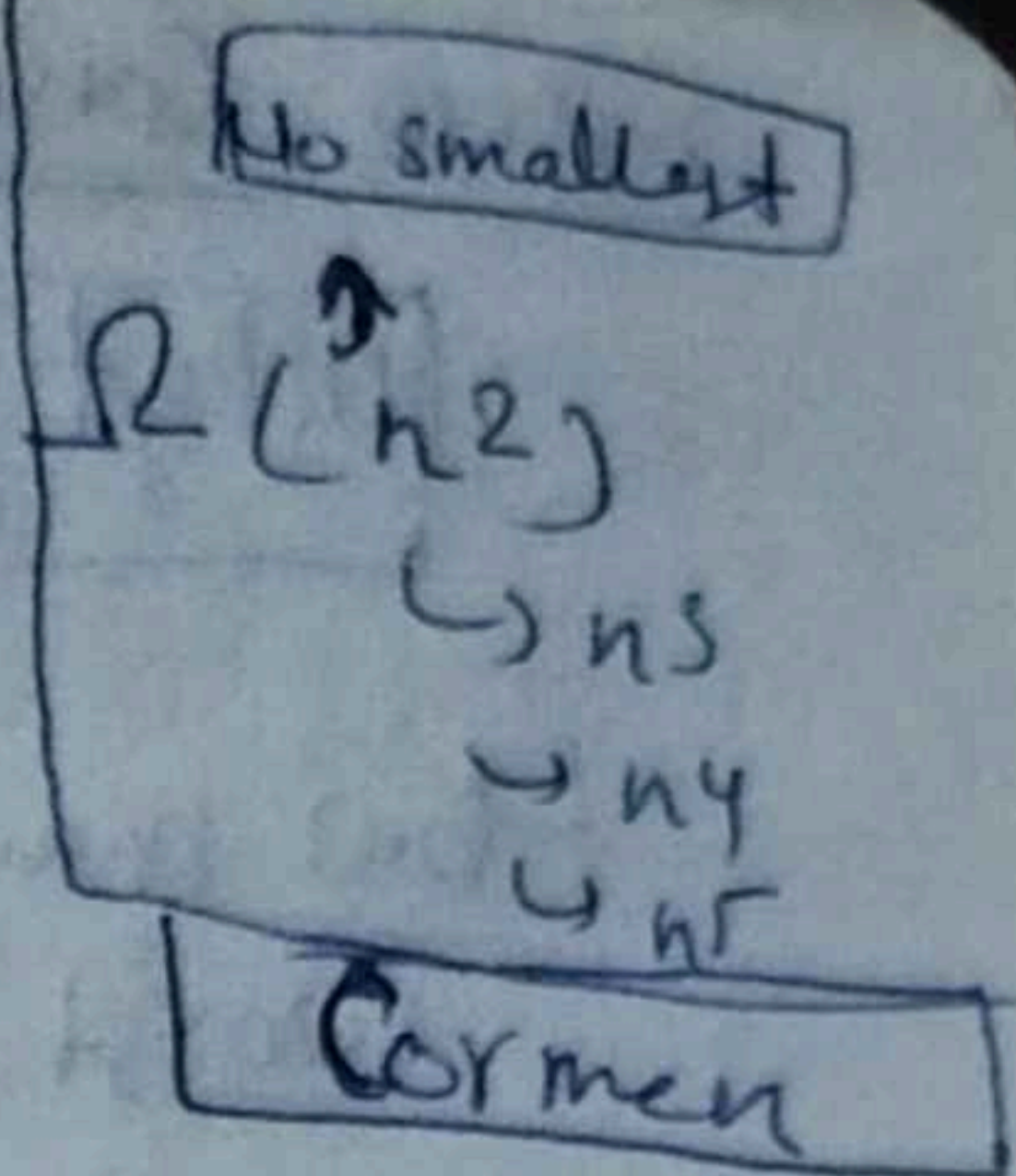
$$\text{time } c = [a]n^2 + [b]n + [c] \text{ (function)}$$

$$T_c = O(n^2)$$

largest term

Big Omega Notation (lower bound)

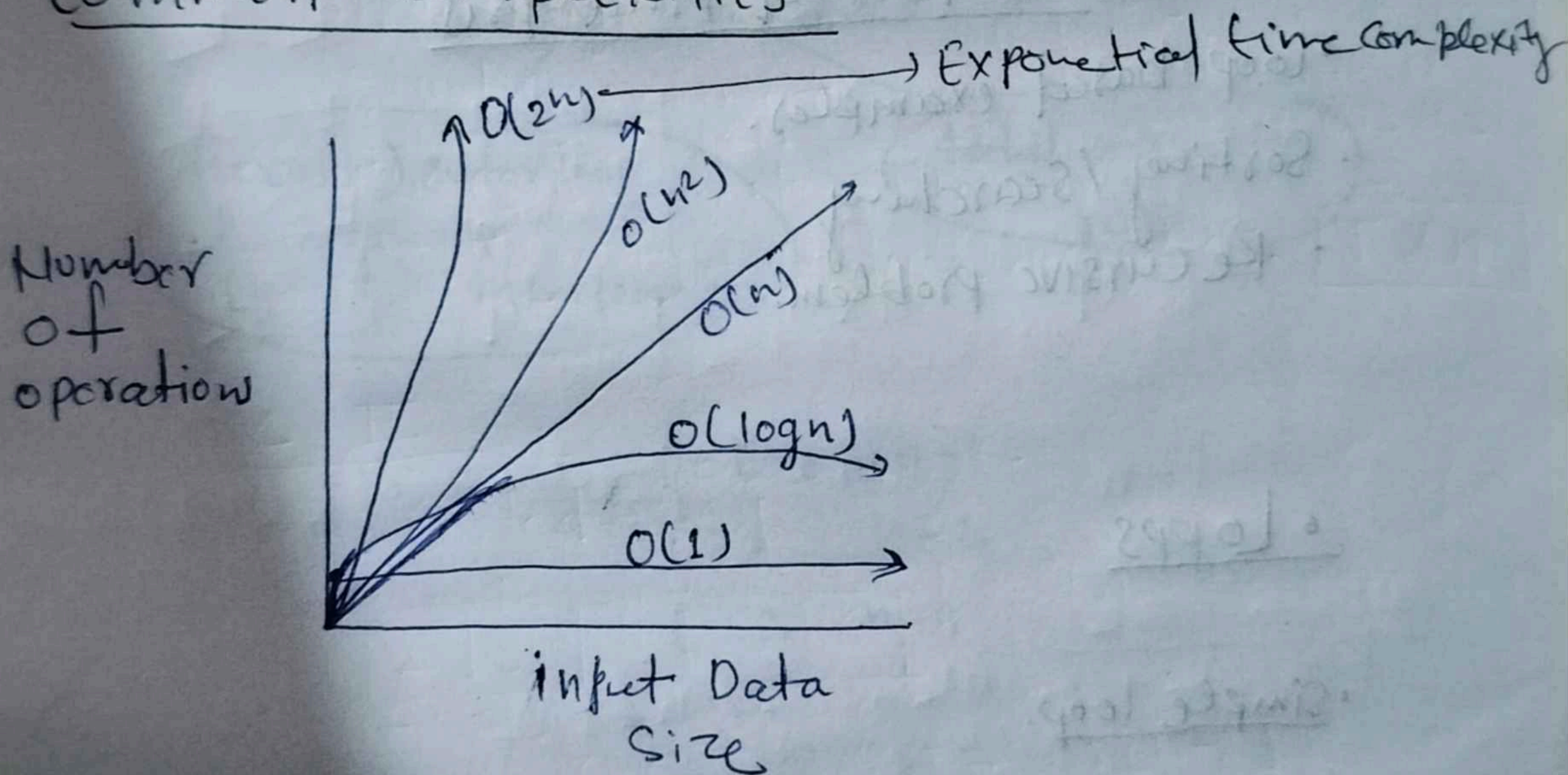
- Best case time complexity



Big Theta (Θ) (Average bound)

- Between of $O()$ and $\Omega()$.

Common Complexities



Space Complexity

Input Space + Auxiliary Space

How many temporary (auxiliary space) we use in worst case.

but most of time we consider / priority given time complexity.

Theoretical Analysis

- Loop based examples.
- Sorting / Searching
- Recursive problem.

Loops

Simple loop.

```
for (int i=0; i<n; i++)  
{  
    Constant work — k  
}
```

$i=0 \rightarrow k$

$i=1 \rightarrow k$

$i=2 \rightarrow k$

$i=3 \rightarrow k$

$\vdots$
 $i=n-1 \rightarrow k$

$\rightarrow O(n \times k)$, Constant ignore

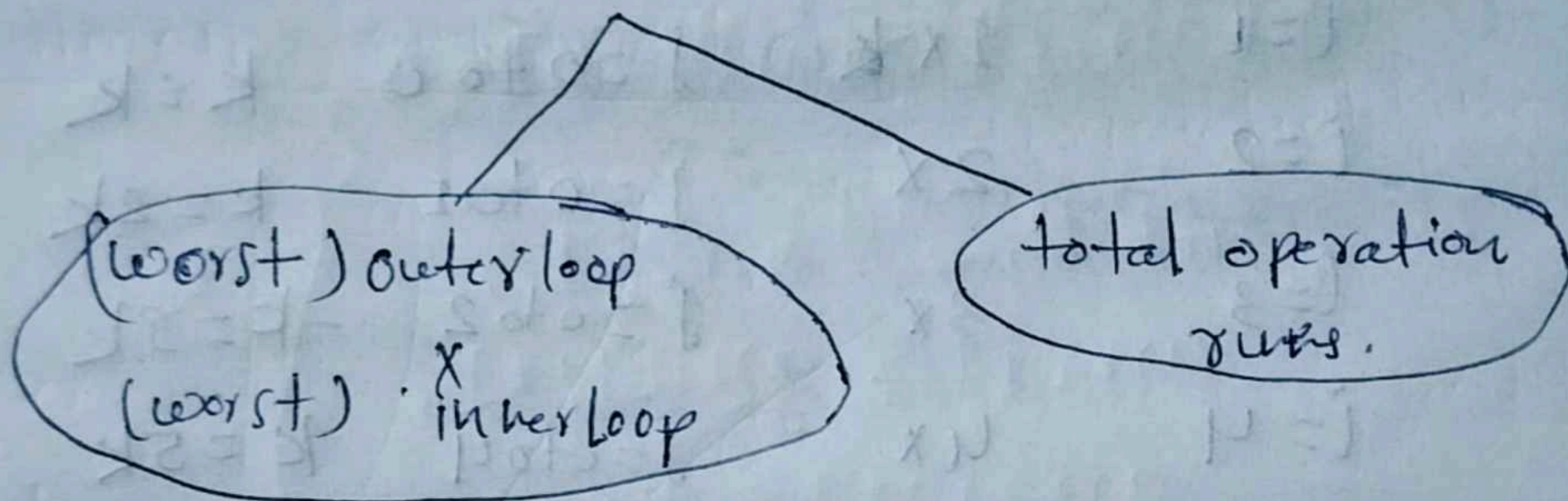
$O(n)$

Nested loop 1

```
for (int i = 0; i < n; i++)  
{  
    for (int j = i + 1; j < n; j++)  
    {
```

} // Some constant work $J \rightarrow k$
}

nested loop



$$i = 0 \quad n-1$$

$$i = 1 \quad n-2$$

$$i = 2 \quad n-3$$

$$i = n-1 \quad 0$$

$$\underbrace{(n-1) + (n-2) + (n-3) + \dots + 1}_{0 \text{ to } n} \quad \swarrow \quad 0 \text{ to } n-1$$

$$\boxed{O(n^2)}$$

$$\Rightarrow \frac{n(n+1)}{2}$$

$$\Rightarrow \frac{n^2 + n}{2}$$

$$\Rightarrow O\left(\frac{n^2}{2} + \frac{n}{2}\right)$$

$$O(n^2) - \text{high term}$$

Nested loop 2

```
for (int i=0; i<n; i++)  
{  
    for (int j=0; j<i; j++)  
    {  
        k  
    }  
}
```

$i = 0 \text{ to } n-1$
 $j = 0 \text{ to } i-1$

$i=0$ 0 times.

$i=1$ 1x ~~k~~ $j=0 \text{ to } 0$ $k=k$

$i=2$ 2x $j=0 \text{ to } 1$ $k=2k$

$i=3$ 3x $j=0 \text{ to } 2$ $k=3k$

$i=4$ 4x $j=0 \text{ to } 3$ $k=5k$

!

$i=n$ $(n-1)$ $j=0 \text{ to } n-1$ $k=(n-1)k$

$$k + 2k + 3k + 4k + \dots + (n-1)k$$

$$k(1+2+3+\dots+n-1)$$

$$k \left(\frac{n(n-1)}{2} \right) \Rightarrow \left(\frac{kn^2}{2} - \frac{kn}{2} \right)$$

Step 1 low order terms

$$O(n^2 - n)$$

Step 2

$$O(n^2)$$

largest term.

$i = 0$ to n

0 to i

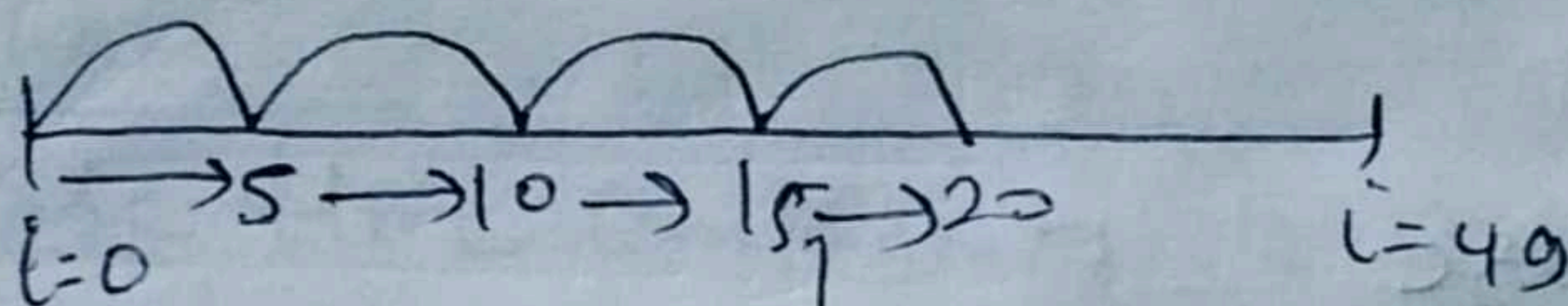
$0 + 1 + 2 + \dots + (n-1)$

$O(n^2)$

Nested loop 3

```
for (int i = 0; i < n; i = i + k)
{
    for (int j = i + 1; j <= k; j++)
    {
        // Some constant work [p]
    }
}
```

$n = 50$, $k = 5$



$$O\left(\frac{n}{k} \times k \times p\right)$$

$$O(np)$$

Constant

$$\rightarrow O(n)$$

worst case. O Start

Sorting Analysis

Bubble Sort

k (constant work)

```
public static void bubbleSort(int arr[])  
{  
    for(int i=0; i<arr.length-1; i++)  
    {  
        for(int j=0; j<arr.length-1-i; j++)  
        {  
            if(arr[j] > arr[j+1])  
            {  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
            }  
        }  
    }  
}
```

worst case

$i=0, j=0$ to $n-1$ Outer loop $-n$
 $i=1, j=0$ to $n-2$ inner loop $- j=0$ to $n-1-i$
| | |
| | |

K

inner loop

$n \times k$
 $(n-1) \times k$
 $(n-2) \times k$

} that is AP

$$K(n + (n-1) + (n-2) + \dots + 1)$$

$$K(n^2)$$

$$\boxed{O(n^2)}$$

Easy to

if n times outer loop
then that inner loop n times
run

like

Outer loop	inner loop	
1	n	$k = nk$
2	$2n$	$\rightarrow 2nk$
3	$3n$	$\rightarrow 3nk$

Best case

Same time complexity
Because loop runs same
time, only skipped if
conditions. So Best time
Complexity is $\boxed{O(n^2)}$

$$\boxed{O(n^2)}$$

$$n \rightarrow n \times n \rightarrow n^2 k$$

According to
Upper code

⊗ But we write Optimized Bubble Sort code
then Change Best case time complexity $\boxed{O(n)}$
worst case time complexity is same $\boxed{O(n^2)}$

Binary Search

```
public static int int binSearch(int arr[], int key)
```

```
{
```

```
    int start = 0;
```

```
    int end = arr.length - 1;
```

```
    while (start <= end)
```

```
    {
```

```
        int mid = (end + start) / 2;
```

```
        if (arr[mid] == key)
```

```
        {
```

```
            return mid;
```

```
        }
```

```
        else if (arr[mid] < key)
```

```
        {
```

```
            start = mid + 1;
```

```
        }
```

```
        else
```

```
        {
```

```
            end = mid - 1;
```

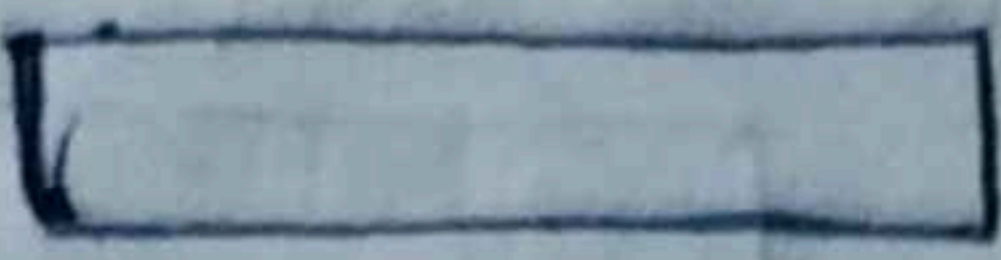
```
        }
```

```
    }
```

```
    return -1;
```

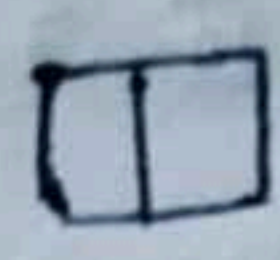
```
}
```


worst case $P = \text{constant work}$

given array size n 

According to Binary Search

$$\frac{n}{2} = \text{$$

again $\frac{n}{4} = \text{$

again $\frac{n}{8} = \text{$

$$\frac{n}{2^k} = 1$$

means array divided whenever array sized is one.

$$\frac{n}{2^k} = 1$$
$$n = 2^k$$

$$\text{loops run} = k$$

$$\log_2 n = k$$

Time complexity is $O(k \times P)$

$$O(\log n \times P)$$

constant

$$O(\log n)$$

Linear Search have a $O(n)$ time complexity
And Binary Search have a $O(\log n)$ So
Binary Search is Better to Linear Search algorithm.

Recursive Algorithms

Linear
factorial (Rec.)

Divide and
Conquer
(Merge sort)

① Total work done = (no of calls * work in each call)

② Recurrence Equation.

Space Complexity = (max depth * memory in each call)

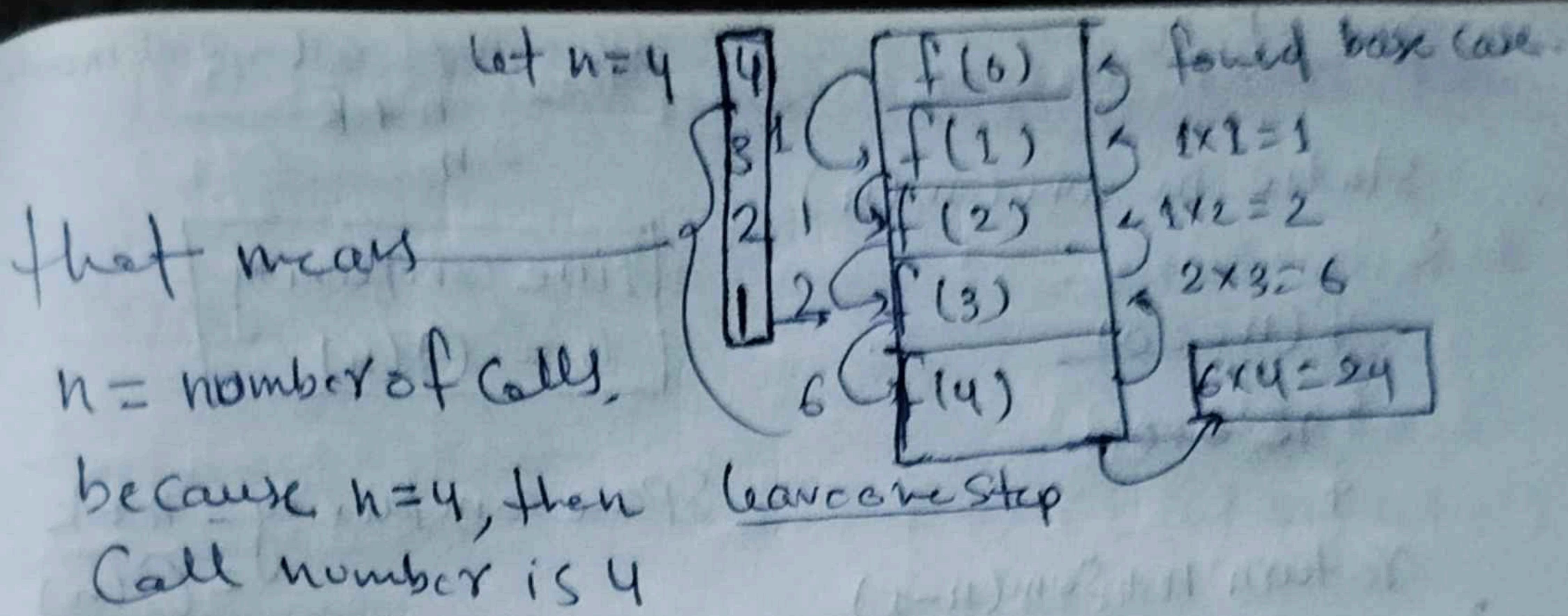
Factorial

```
public static int fact (int n)
{
```

```
    if (n == 0) { — K1
    {
        return 1;
    }
```

```
    return n * fact (n-1); — K2
```

```
}
```

$$WD = \text{no of calls} * \text{work in each call}$$

In factorial work in each call is k_1 and k_2

So.

$$WD = n * [k_1, k_2]$$

ignore constant

$$T_m = [O(n)]$$

Space complexity = max depth * each level mem.

$$= n * k$$

$$\Rightarrow O(n)$$

constant memory.

Sum of n

```
static int sum(int n)
{
    if (n == 0)
    {
        return 0;
    }
    return n + sum(n-1);
}
```

$$\begin{aligned} \text{WD} &= \text{no of call} + \text{work in each call} \\ &= n * k \\ &\Rightarrow n \end{aligned}$$

Time Complexity
is $= O(n)$

$$\begin{aligned} \text{Space complexity} &= n * k \\ &= O(n) \end{aligned}$$

k is constant memory
because no extra memory
use.

Fibonacci

```
public class fibonacci
{
    static int fib(int n)
    {
        if (n == 0 || n == 1)
        {
            return n;
        }
        return fib(n-1) + fib(n-2);
    }
}
```

Depth first
approach

$$SC = n * O(1)$$

$$SC = O(n)$$

$$TC = O(2^n)$$

- each level have two choice
total level = n

$$\text{So } \Rightarrow 2^n$$

Detail in
Backtracking
Chapter

Merge Sort (OAC)

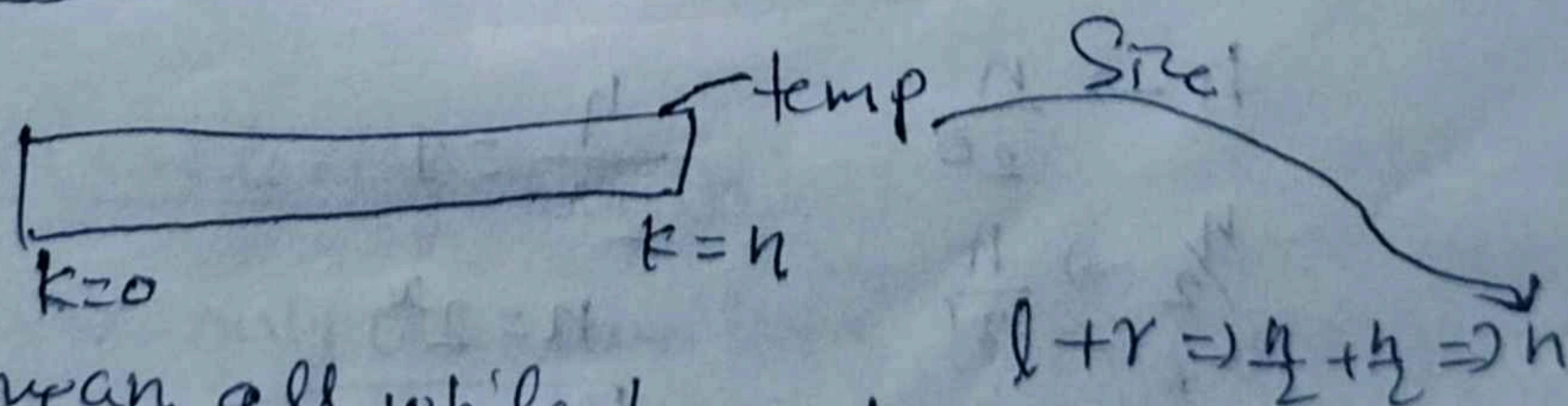
```

public void mergesort(int arr[], int si, int ei)
{
    if (si >= ei)
    {
        return;
    }
    int mid = si + (ei - si) / 2;
    mergesort(arr, si, mid);
    mergesort(arr, mid + 1, ei);
    merge(arr, si, mid, ei);
}
    
```

• merge function

first of all make temp array size of both
divid array

$\boxed{n/2} \quad \boxed{n/2} \rightarrow$ worst case



• that mean all while loop time
Complexity is $O(n)$

• copy loop time complexity is $O(n)$

that means merge function have $T_c = O(n)$
 $S_c = O(n)$

now ~~merge~~ merge sort

$$f(n) = 2f(n/2)$$

left + Right part
 + n work merge
 function.

$$T(n) = T(n/2) + T(n/2) + (n+k) \quad (\text{Recurrence Relation})$$

$$T(n) = T(n/2) + T(n/2) + nk$$

$$T(n) = 2T(n/2) + nk$$

$$2 \times [T(n/2) = 2T(n/2) + \frac{n}{2} \cdot k] \rightarrow nk$$

$$4 \times [T(n/4) = 2T(n/4) + \frac{n}{4} \cdot k] \rightarrow nk$$

$$8 \times [T(n/8) = 2T(n/8) + \frac{n}{8} \cdot k] \rightarrow nk$$

$$T(1) = O(1)$$

$$nk$$

$$n \rightarrow \frac{n}{2^0}$$

$$\frac{n}{2^k} = 1$$

$$\frac{n}{2} \rightarrow \frac{n}{2^1}$$

$$n = 2^k$$

$$1 \rightarrow \frac{n}{2} \cdot k$$

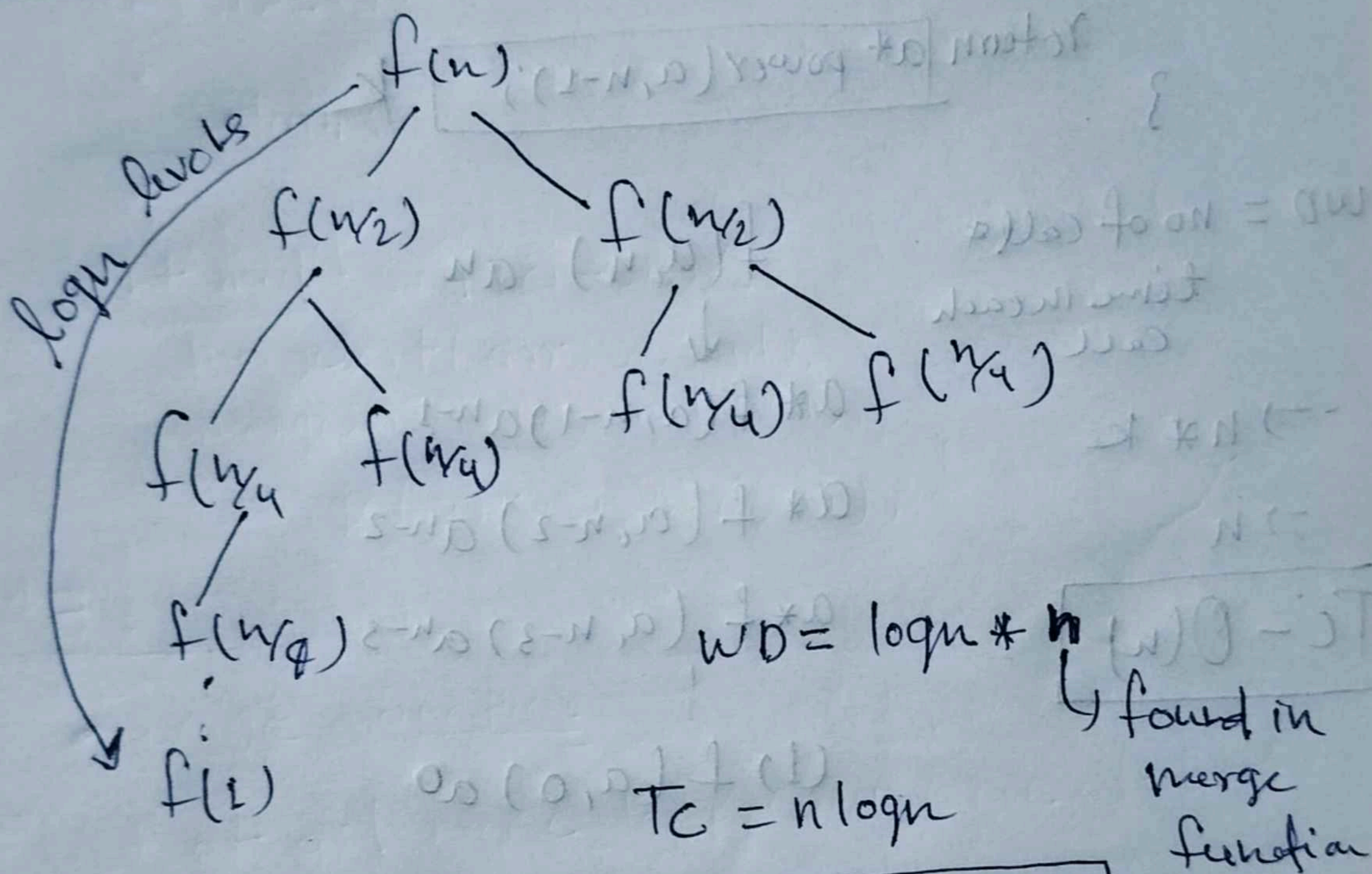
$$\boxed{\log_2 n = k}$$

$$T(n) \Rightarrow O(1) + (\log n)(n)$$

$$T_c = n \log n$$

$$T_c = O(n \log n)$$

● Another



$$\text{Time complexity} = O(n \log n)$$

$$\text{Space complexity} = O(n)$$

Because only one auxiliary array use in merge function. and constant.

Power Function 1

```
Public Static int power (int a, int n)
```

```
{
```

```
    if (n == 0)
```

```
    {
```

```
        return 1;
```

```
    }
```

```
    return a * power(a, n-1);
```

```
}
```

WD = no of call *
time in each
call

$\Rightarrow n * k$

$\Rightarrow n$

Tc - $O(n)$

$f(a, n) = a^n$

↓

$a * f(a, n-1) = a^n$

$a * f(a, n-2) = a^{n-2}$

$a * f(a, n-3) = a^{n-3}$

(1) $f(a, 0) = a^0$

Time complexity = $O(n)$

Space complexity = $O(1)$

For each recursive call, we are not using any extra space. We are only using the space for the function call stack.